



Faculty of Information Technology

Fall- 2025

Introduction to Computers Programming CS.102

By Dr. Doha Taha

Introduction to C++ Programming, Input/ Output and Operators

Chapter 2 of C++ How to Program

First Program In C++: Printing A Line Of Text

```
1 // Fig. 2.1: fig02_01.cpp
2 // Text-printing program.
3 #include <iostream> // enables program to output data to the screen
4
5 // function main begins program execution
6 int main() {
7     std::cout << "Welcome to C++!\n"; // display message
8
9     return 0; // indicate that program ended successfully
10 }
```

Welcome to C++!

Fig. 2.1 | Text-printing program.



First Program In C++: Printing A Line Of Text (Cont.)

✧ // indicates that the remainder of each line is a **comment**.

- ❑ You insert comments to document your programs and to help other people read and understand them.
- ❑ The C++ compiler ignores comments and do not cause any machine-language object code to be generated.

✧ A comment beginning with // is called a **single-line comment** because it terminates at the end of the current line.

✧ You also may use comments containing one or more lines enclosed in **/* and */**.



First Program In C++: Printing A Line Of Text (Cont.)

- ✧ A **preprocessing directive** is a message to the C++ preprocessor.
- ✧ Lines that begin with **#** are processed by the preprocessor before the program is compiled.
- ✧ `#include <iostream>` notifies the preprocessor to include in the program the contents of the **input/output stream header file <iostream>**.
- ❑ This header is a file containing information used by the compiler when compiling any program that outputs data to the screen or inputs data from the keyboard using C++-style stream input/output.



First Program In C++: Printing A Line Of Text (Cont.)

✧ You use blank lines, *space characters* and *tab characters* (i.e., “tabs”) to make programs easier to read.

- ❑ Together, these characters are known as **white space**.
- ❑ White-space characters are normally *ignored* by the compiler.



First Program In C++: Printing A Line Of Text (Cont.)

- ▶ **main** is a part of every C++ program.
- ▶ The parentheses after main indicate that **main** is a program building block called a **function**.
- ▶ C++ programs typically **consist of** one or more functions and classes.
- ▶ Exactly **one** function in every program *must* be named main.
- ▶ C++ programs begin **executing** at the function main, even if main is *not* the first function defined in the program.
- ▶ The keyword **int** to the left of main indicates that main “**returns**” an integer (whole number) value.
 - For now, simply include the keyword **int** to the left of **main** in each of your programs.



First Program In C++: Printing A Line Of Text (Cont.)

- ✧ A **left brace**, `{`, must *begin* the **body** of every function.
- ✧ A corresponding **right brace**, `}`, must *end* each function's body.
- ✧ A statement instructs the computer to **perform an action**.
- ✧ Together, the quotation marks and the characters between them are called a **string**, a **character string** or a **string literal**.
- ✧ We refer to characters between double quotation marks simply as **strings**.
 - ... White-space characters in strings are not ignored by the compiler.
- ✧ Most C++ statements end with a **semicolon** (`;`), also known as the **statement terminator**.
 - ... Preprocessing directives (like `#include`) do not end with a semicolon.



First Program In C++: Printing A Line Of Text (Cont.)

- ✧ Typically, output and input in C++ are accomplished with **streams** of data.
- ✧ When a `cout` statement executes, it sends a stream of characters to the **standard output stream object**—`std::cout`—which is normally “connected” to the screen.
- ✧ The `std::` before `cout` is required when we use names that we’ve brought into the program by the preprocessing directive `#include <iostream>`.
 - ❑ The notation `std::cout` specifies that we are using a name, in this case `cout`, that belongs to “namespace” `std`.
 - ❑ The names `cin` (the standard input stream) and `cerr` (the standard error stream) also belong to namespace `std`.



First Program In C++: Printing A Line Of Text (Cont.)

- ✧ In the context of an output statement, the << operator is referred to as the **stream insertion operator**.
 - ❑ The value to the operator's right, the right **operand**, is inserted in the output stream.
- ✧ The characters **\n** are *not* printed on the screen.
- ✧ The backslash (****) is called an **escape character**.
 - ❑ It indicates that a “special” character is to be output.
- ✧ When a backslash is encountered in a string of characters, the next character is combined with the backslash to form an **escape sequence**.
- ✧ The escape sequence **\n** means **newline**.
 - ❑ Causes the **cursor** to move to the beginning of the next line on the screen.



Escape sequence	Description
<code>\n</code>	Newline. Position the screen cursor to the beginning of the next line.
<code>\t</code>	Horizontal tab. Move the screen cursor to the next tab stop.
<code>\r</code>	Carriage return. Position the screen cursor to the beginning of the current line; do not advance to the next line.
<code>\a</code>	Alert. Sound the system bell.
<code>\\</code>	Backslash. Used to print a backslash character.
<code>\'</code>	Single quote. Used to print a single-quote character.
<code>\"</code>	Double quote. Used to print a double-quote character.



First Program In C++: Printing A Line Of Text (Cont.)

- ✧ When the **return statement** is used at the end of `main`, the value `0` indicates that the program has *terminated successfully*.
- ✧ According to the C++ standard, if program execution reaches the end of `main` without encountering a `return` statement, it's assumed that the program terminated successfully—exactly as when the last statement in `main` is a **return** statement with the value `0`.

```
1 // Fig. 2.3: fig02_03.cpp
2 // Printing a line of text with multiple statements.
3 #include <iostream> // enables program to output data to the screen
4
5 // function main begins program execution
6 int main() {
7     std::cout << "Welcome ";
8     std::cout << "to C++!\n";
9 } // end function main
```

Welcome to C++!

Fig. 2.3 | Printing a line of text with multiple statements.



Modifying Our First C++ Program (Cont.)

- ✧ A single statement can print multiple lines by using newline characters.
- ✧ Each time the `\n` (newline) escape sequence is encountered in the output stream, the screen cursor is positioned to the beginning of the next line.
- ✧ To get a blank line in your output, place two newline characters back to back.

```
1 // Fig. 2.4: fig02_04.cpp
2 // Printing multiple lines of text with a single statement.
3 #include <iostream> // enables program to output data to the screen
4
5 // function main begins program execution
6 int main() {
7     std::cout << "Welcome\nto\n\nC++!\n";
8 } // end function main
```

```
Welcome
to

C++!
```

Fig. 2.4 | Printing multiple lines of text with a single statement.



Another C++ Program: Adding Integers

- ✧ The next program obtains two integers typed by a user at the keyboard, computes their sum and outputs the result using **std::cout**.
- ✧ The Next Figure shows the program and sample inputs and outputs.



```
2 // Addition program that displays the sum of two integers.
3 #include <iostream> // enables program to perform input and output
4
5 // function main begins program execution
6 int main() {
7     // declaring and initializing variables
8     int number1{0}; // first integer to add (initialized to 0)
9     int number2{0}; // second integer to add (initialized to 0)
10    int sum{0}; // sum of number1 and number2 (initialized to 0)
11
12    std::cout << "Enter first integer: "; // prompt user for data
13    std::cin >> number1; // read first integer from user into number1
14
15    std::cout << "Enter second integer: "; // prompt user for data
16    std::cin >> number2; // read second integer from user into number2
17
18    sum = number1 + number2; // add the numbers; store result in sum
19
20    std::cout << "Sum is " << sum << std::endl; // display sum; end line
21 }
```

Fig. 2.5 | Addition program that displays the sum of two integers.

OUT PUT

```
Enter first integer: 45  
Enter second integer: 72  
Sum is 117
```

Fig. 2.5 | Addition program that displays the sum of two integers.



Another C++ Program: Adding Integers (Cont.)

- ✧ **Declarations** introduce identifiers into programs.
- ✧ The identifiers `number1`, `number2` and `sum` are the names of **variables**.
- ✧ A variable is a location in the computer's memory where a value can be stored for use by a program.
- ✧ Variables `number1`, `number2` and `sum` are data of type **int**, meaning that these variables will hold **integers** (whole numbers such as 7, -11, 0 and 31914).

Another C++ Program: Adding Integers (Cont.)

✧ Lines 8–10 initialize each variable to 0 by placing a value in braces (**{** and **}**) immediately following the variable's name

- ❑ Known as list initialization

- ❑ Introduced in C++11

✧ Previously, these declarations would have been written as:

- ❑ `int number1 = 0;`

- ❑ `int number2 = 0;`

- ❑ `int sum = 0;`

```
8      int number1{0}; // first integer to add (initialized to 0)
9      int number2{0}; // second integer to add (initialized to 0)
10     int sum{0}; // sum of number1 and number2 (initialized to 0)
11
```

Another C++ Program: Adding Integers (Cont.)

- ⌘ All variables *must* be declared with a ***name*** and a ***data type*** before they can be used in a program.
- ⌘ If more than one name is declared in a declaration (as shown here), the names are separated by commas (,); this is referred to as a **comma-separated list**.

Another C++ Program: Adding Integers (Cont.)

- ✧ Data type **double** is for specifying real numbers, and data type **char** for specifying *character data*.
- ✧ Real numbers are numbers with decimal points, such as 2.5, 0.0 and -13.5.
- ✧ A **char** variable may hold only a single lowercase letter, a single uppercase letter, a single digit or a single special character (e.g., \$ or *).
- ✧ Types such as **int**, **double** and **char** are called **fundamental types**.
- ✧ Fundamental-type names are keywords and therefore *must* appear in all **lowercase** letters.
- ✧ Appendix C contains the complete list of fundamental types.

Another C++ Program: Adding Integers (Cont.)

- ⌘ A variable name is any valid **identifier** that is *not* a keyword.
- ⌘ An identifier is a series of characters consisting of letters, digits and underscores (`_`) that does not begin with a digit.
- ⌘ C++ is **case sensitive**—uppercase and lowercase letters are different, so `a1` and `A1` are *different* identifiers.

Another C++ Program: Adding Integers (Cont.)

⌘ Declarations of variables can be placed almost anywhere in a program, but they must appear *before* their corresponding variables are used in the program.

Another C++ Program: Adding Integers (Cont.)

- ✧ A **prompt** it directs the user to take a specific action.
- ✧ A **cin** statement uses the **input stream object cin** (of namespace **std**) and the **stream extraction operator, >>**, to obtain a value from the keyboard.
- ✧ Using the stream extraction operator with **std::cin** takes character input from the standard input stream, which is usually the keyboard.

Another C++ Program: Adding Integers (Cont.)

- ✧ When the computer executes an input statement that places a value in an `int` variable, it waits for the user to enter a value for variable `number1`.
- ✧ The user responds by typing the number (as characters) then pressing the *Enter* key (sometimes called the *Return* key) to send the characters to the computer.
- ✧ The computer converts the character representation of the number to an integer and assigns (i.e., copies) this number (or **value**) to the variable `number1`.
- ✧ Any subsequent references to `number1` in this program will use this same value.
- ✧ Pressing *Enter* also causes the cursor to move to the beginning of the next line on the screen.

Another C++ Program: Adding Integers (Cont.)

✎ In this program, an assignment statement adds the values of variables `number1` and `number2` and assigns the result to variable `sum` using the **assignment operator** `=`.

... Most calculations are performed in assignment statements.

✎ The `=` operator and the `+` operator are called **binary operators** because each has two operands.

Another C++ Program: Adding Integers (Cont.)

- ✧ `std::endl` is a so-called **stream manipulator**.
- ✧ The name `endl` is an abbreviation for “end line” and belongs to namespace `std`.
- ✧ The `std::endl` stream manipulator outputs a newline, then “flushes the output buffer.”
 - ... This simply means that, on some systems where outputs accumulate in the machine until there are enough to “make it worthwhile” to display them on the screen, `std::endl` forces any accumulated outputs to be displayed at that moment.
 - ... This can be important when the outputs are prompting the user for an action, such as entering data.

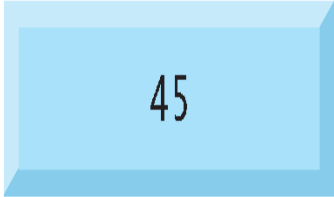
Another C++ Program: Adding Integers (Cont.)

- ✧ Using multiple stream insertion operators (<<) in a single statement is referred to as **concatenating**, **chaining** or **cascading stream insertion operations**.
- ✧ Calculations can also be performed in output statements.

Memory Concepts

- ✧ Variable names such as number1, number2 and sum actually correspond to **locations** in the computer's memory.
- ✧ Every variable has a name, a type, a size and a value.
- ✧ When a value is placed in a memory location, the value overwrites the previous value in that location; thus, placing a new value into a memory location is said to be **destructive**.
- ✧ When a value is read out of a memory location, the process is **nondestructive**.

number1



45

Fig. 2.6 | Memory location showing the name and value of variable number1.



number1

45

number2

72

Fig. 2.7 | Memory locations after storing values in the variables for number1 and number2.



number1	45
number2	72
sum	117

Fig. 2.8 | Memory locations after calculating and storing the sum of number1 and number2.



Arithmetic

- ✧ Most programs perform arithmetic calculations.
- ✧ Figure 2.9 summarizes the C++ **arithmetic operators**.
- ✧ The **asterisk** (*) indicates multiplication.
- ✧ The **percent sign** (%) is the remainder operator.
 - ❑ Yields the remainder after integer division.
 - ❑ Can be used only with integer operands.
- ✧ The arithmetic operators are all binary operators.
- ✧ **Integer division** (i.e., where both the numerator and the denominator are integers) yields an integer quotient.
 - ... Any fractional part in integer division is discarded (i.e., **truncated**)—no rounding occurs.

Operation	Arithmetic operator	Algebraic expression	C++ expression
Addition	+	$f + 7$	<code>f + 7</code>
Subtraction	-	$p - c$	<code>p - c</code>
Multiplication	*	bm or $b \cdot m$	<code>b * m</code>
Division	/	x / y or $\frac{x}{y}$ or $x \div y$	<code>x / y</code>
Remainder	%	$r \bmod s$	<code>r % s</code>

Fig. 2.9 | Arithmetic operators.



ARITHMETIC (CONT.)

- ✧ Arithmetic expressions in C++ must be entered into the computer in **straight-line form**.
- ✧ Expressions such as “a divided by b” must be written as a / b , so that all constants, variables and operators appear in a straight line.
- ✧ Parentheses are used in C++ expressions in the same manner as in algebraic expressions.
- ✧ For example, to multiply a times the quantity $b + c$ we write

... $a * (b + c)$

ARITHMETIC (CONT.)

✧ C++ applies the operators in arithmetic expressions in a precise sequence determined by the following **rules of operator precedence**, which are generally the same as those followed in algebra.

Operator(s)	Operation(s)	Order of evaluation (precedence)
()	Parentheses	Evaluated first. For <i>nested</i> parentheses, such as in the expression $a * (b + c / (d + e))$, the expression in the <i>innermost</i> pair evaluates first. [<i>Caution:</i> If you have an expression such as $(a + b) * (c - d)$ in which two sets of parentheses are not nested, but appear “on the same level,” the C++ Standard does <i>not</i> specify the order in which these parenthesized subexpressions will evaluate.]
* / %	Multiplication Division Remainder	Evaluated second. If there are several, they’re evaluated left to right.
+ -	Addition Subtraction	Evaluated last. If there are several, they’re evaluated left to right.

Fig. 2.10 | Precedence of arithmetic operators.



ARITHMETIC (CONT.)

- ⌘ There is no arithmetic operator for exponentiation in C++, so x^2 is represented as $x * x$.
- ⌘ Figure 2.11 illustrates the order in which the operators in a second-degree polynomial are applied.
- ⌘ As in algebra, it's acceptable to place **redundant parentheses** in an expression to make the expression clearer.

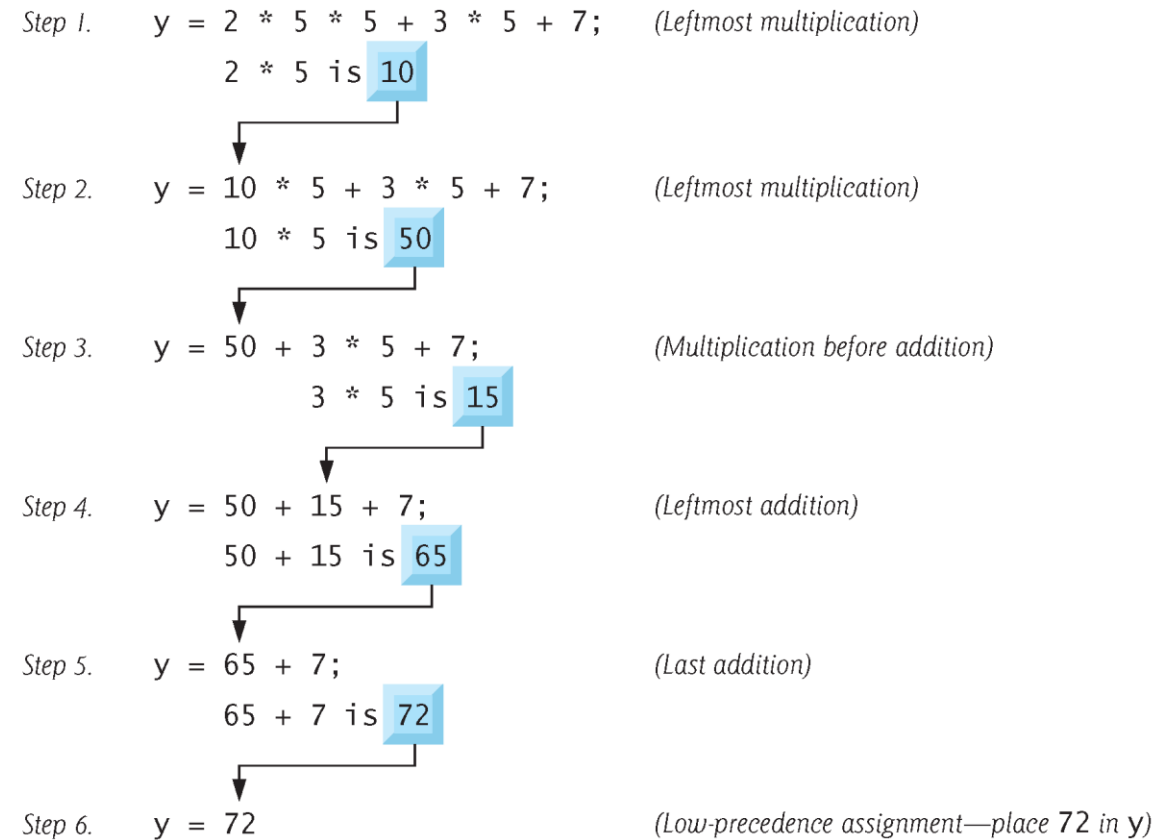


Fig. 2.11 | Order in which a second-degree polynomial is evaluated.



Decision Making: Equality And Relational Operators (Cont.)

- ✧ Fig. 2.12 uses six `if` statements to compare two numbers input by the user.
- ✧ If the condition in any of these `if` statements is satisfied, the output statement associated with that `if` statement executes.
- ✧ Figure 2.13 shows the program and the input/output dialogs of three sample executions.

What is the output of the following program?

```
#include <iostream>
#include <conio>
main()
{
    int a,b,c;
    a=1;
    b=2;
    c=a==b;
    cout << c;
    getch();
}
```

0

F

What is the output of the following program?

```
#include <iostream>
#include <conio>
main()
{
    int a,b,c;
    a=2;
    b=2;
    c=a==b;
    cout << c;
    getch();
}
```

1

T

What is the output of the following program?

```
#include <iostream>
#include <conio>
main()
{
    int a,b,c;
    a=2;
    b=2;
    c=a!=b;
    cout << c;
    getch();
}
```

0

What is the output of the following program?

```
#include <iostream>
#include <conio>
main()
{
    int a,b,c;
    a=1;
    b=2;
    c=a<b;
    cout << c;
    getch();
}
```

1

What is the output of the following program?

```
#include <iostream>
#include <conio>
main()
{
    int a,b,c;
    a=2;
    b=2;
    c=a<=b;
    cout << c;
    getch();
}
```

1

Test >

Test >=

What is the output of the following program?

```
#include <iostream>
#include <conio>
main()
{
    int a,b;
    a=2;
    b=2;

    if (a==b)
        cout << endl << "1";
    cout << endl << "2";
    cout << endl << "3";

    getch();
}
```

1
2
3



What is the output of the following program?

```
#include <iostream>
#include <conio>
main()
{
    int a,b;
    a=1;
    b=2;

    if (a==b)
        cout << endl << "1";
    cout << endl << "2";
    cout << endl << "3";

    getch();
}
```

2
3



What is the output of the following program?

```
main()
{
    int a,b;
    a=1;
    b=2;

    if (a==b)
    {
        cout << endl << "1";
        cout << endl << "2";
    }
    cout << endl << "3";

    getch();
}
```

3

THANK YOU